PROJE-1:DOSYA BÜTÜNLÜĞÜ

Sema Koç - Gedik Üniversitesi Bilişim Güvenliği Teknolojisi

AMAÇ

Temel amacı, bilgisayarımızdaki bir dosyanın sonradan değiştirilip değiştirilmediğini kontrol etmekti. Örneğin `notlar.txt` isimli bir dosyan olsun. Program ilk çalıştırıldığında dosyanın bir çeşit dijital parmak izini oluşturuyor. Daha sonra tekrar çalıştırdığında bu parmak izini karşılaştırıyor.

1-Gerekli modüller

```
import hashlib
from pathlib import Path
```

`Import:`Python'da daha önceden hazırlanmış araçları kodumuzda kullanmamızı sağlar.

- `hashlib`: SHA-256 gibi hash algoritmalarını kullanmamızı sağlar.
- `Path`: Dosya yollarını daha kolay yönetmemizi sağlar. Örneğin dosya var mı, dosyanın adı ne, tam yolu ne gibi işlemlerde kullanırız.

Neden bunları kullandık?:Çünkü dosyanın hash değerini hesaplamamız ve dosya yoluyla ilgili işlemleri yapmamız gerekiyordu.

2-Fonksiyon oluşturmak

```
def dosya_hash_hesapla(dosya_yolu):
    sha256 = hashlib.sha256()
```

1.`def:` Python'da bir fonksiyon tanımlamak için kullanılır. Fonksiyon, belirli bir işi yapan ve gerektiğinde tekrar kullanabileceğimiz kod grubudur.

Bizim fonksiyonumuzun adı: `dosya_hash_hesapla`

Yani bu fonksiyonun görevi, verilen dosyanın hash değerini hesaplamak.

Parantez içindeki `dosya_yolu` ise fonksiyona vereceğimiz dosyanın yolunu temsil eder. Buna parametre denir.

2. `hashlib.sha256():`SHA-256 hesaplamasını başlatacak bir nesne oluşturuyor.

3-Dosyayı açmak ve okumak

```
with open(dosya_yolu, "rb") as dosya:
    while True:
        parca = dosya.read(4096)

        if not parca:
            break

        sha256.update(parca)
```

`open(dosya_yolu, "rb")` ->Dosyayı açar.

- `dosya_yolu`: Hangi dosyayı açacağımızı belirtir.
- `"rb"`: Dosyayı ikili veri (binary) olarak okumamızı sağlar. Bu, yalnızca metin dosyalarıyla sınırlı kalmadan farklı dosya türlerini de işleyebilmemize yardımcı olur.

`with ... as dosya` ->Dosyayla güvenli biçimde çalışmamızı sağlar. İşimiz bittiğinde dosya otomatik olarak kapatılır.

`while True` ->Bir döngüdür. İçindeki işlemleri, döngüden çıkılana kadar tekrarlar.

`dosya.read(4096)` ->Dosyadan bir seferde en fazla 4096 baytlık veri okur.

Dosyanın tamamını tek seferde belleğe yüklemek yerine parça parça okumak, büyük dosyalarda daha kullanışlıdır.

`if not parca` ->Okunacak veri kalmadığında `parca` boş olur. Python'da boş bir veri bu koşulda doğru kabul edilmez.

`Break`->Döngüyü bitirir.

`sha256.update(parca)` ->Okuduğumuz parçayı hash hesaplamasına ekler. Dosyanın bütün parçaları işlendiğinde dosyanın hash hesabı tamamlanmış olur.

4-Hash sonucunu döndürmek

```
return sha256.hexdigest()
```

Buradaki `return`, fonksiyonun ürettiği sonucu geri verir.

`hexdigest()` : hash sonucunu okunabilir onaltılık karakterlerden oluşan bir metne dönüştürür.

Yani fonksiyonumuz dosyayı okur, hash değerini hesaplar ve bu değeri programın devamına gönderir.

5-Kullanıcıdan dosya yolunu almak

```
dosya_yolu = input(
    "Kontrol edilecek dosyanin yolunu gir: "
).strip().strip('"')

dosya = Path(dosya_yolu)
```

`input()->`Kullanıcıdan bilgi alır. Program çalışınca ekrana bir soru gelir ve kullanıcı cevap yazar.

Örneğin-->`Kontrol edilecek dosyanin yolunu gir:`

`C:\Users\semak\Desktop\notlar.txt`

`.strip()->`Metnin başındaki ve sonundaki gereksiz boşlukları temizler.

`.strip('"')->`Dosya yolu kopyalanırken başına ve sonuna eklenmiş çift tırnakları temizler. Böylece kullanıcı yolu tırnaklarla birlikte yapıştırsa bile program daha rahat işleyebilir.

`Path(dosya_yolu)->`Girilen metni dosya yollarını yönetebileceğimiz bir `Path` nesnesine dönüştürür.

6-Kayıt dosyasını belirlemek

Bu satır önemli, çünkü programın daha önce hesapladığı hash **değerini nerede saklayacağını belirliyor.** Böylece kayıt dosyası, programı hangi klasörden çalıştırdığımıza bağlı olarak başka bir yerde aranmaz; kod dosyasının bulunduğu klasör esas alınır.

```
kayit_dosyasi = Path(__file__).with_name("hash_kaydi.txt")
```

- `__file__`: Çalışan Python dosyasının yolunu belirtir.
- `Path(__file__)`: Bu yolu `Path` nesnesine dönüştürür.
- `.with_name("hash_kaydi.txt")`: Python dosyasının bulunduğu klasörde `hash_kaydi.txt` adlı dosyanın yolunu oluşturur.

7-Dosya gerçekten var mı ?

```
if not dosya.is_file():
    print("Hata: Dosya bulunamadi!")
```

- `if`: Eğer demektir.
- `not`: Değil anlamına gelir.
- `is_file()`: Belirtilen yolun mevcut bir dosyaya ait olup olmadığını kontrol eder.
- `print()`: Ekrana mesaj yazdırır.

Yani kullanıcının yazdığı yolda bir dosya bulunamazsa program hata mesajı gösterir.

Örneğin yanlışlıkla `C:\Users\semak\Desktop\olmayan.txt` yazarsan program dosyayı bulamadığını bildirir.

8-Hash değerini hesaplamak

```
else:
    hash_degeri = dosya_hash_hesapla(dosya)
    tam_yol = str(dosya.resolve())

    print("\nDosya adi:", dosya.name)
    print("SHA-256:", hash_degeri)
```

`else`, önceki `if` koşulu gerçekleşmediğinde çalışır. Yani dosya mevcutsa buraya geliriz.

`dosya_hash_hesapla(dosya):` Az önce oluşturduğumuz fonksiyonu çağırır ve dosyanın hash değerini hesaplar.

`dosya.resolve():` Dosyanın tam yolunu elde etmemize yardımcı olur.

`str(...):` Bu yolu metne dönüştürür.

`Dosya.name:` Dosyanın yalnızca adını verir. Örneğin `notlar.txt`.

`print():` Dosyanın adını ve hash değerini ekrana yazdırır.

9-İlk kontrolde hash değerini kaydetmek

```
if not kayit_dosyasi.exists():
    kayit_dosyasi.write_text(
        tam_yol + "\n" + hash_degeri,
        encoding="utf-8"
    )
    print("\nIlk kontrol tamamlandi.")
    print("Dosyanin referans hash degeri kaydedildi.")
```

Bu bölümün amacı, dosyanın ilk hash değerini referans olarak saklamaktır.

`kayit_dosyasi.exists():` `hash_kaydi.txt` dosyası var mı diye kontrol eder.`not` kullandığımız için koşul, kayıt dosyası mevcut değilse gerçekleşir.

`write_text():` Dosyaya metin yazar. Biz de dosyanın yolunu ve hash değerini kaydediyoruz.

`"\n":` Metin içinde yeni satır anlamına gelir. Böylece dosya yolu ve hash değeri alt alta yazılır.

`encoding="utf-8":` Metnin hangi karakter kodlamasıyla yazılacağını belirtir.

10-Önceden kayıt varsa bilgileri okumak

```
else:
    kayitli_bilgiler = kayit_dosyasi.read_text(
        encoding="utf-8"
    ).splitlines()
```

Bu `else`, kayıt dosyası zaten varsa çalışır.

- `read_text()`: Kayıt dosyasındaki metni okur.
- `splitlines()`: Metni satırlara ayırır ve bir liste oluşturur.

11- Kayıt bozuk mu, başka dosyaya mı ait?

```
if len(kayitli_bilgiler) < 2:
    print("\nHata: Hash kaydi eksik veya bozuk.")
```

`len()`, bir listenin kaç elemanı olduğunu söyler.

Kayıt dosyasında en az iki satır bulunmasını bekliyoruz. Daha azı varsa kayıt eksik olabilir.

Ardından:

```
elif kayitli_bilgiler[0] != tam_yol:
    print("\nHata: Kayit baska bir dosyaya ait.")
    print("Ilk kullandigin dosyanin yolunu gir.")
```

Burada:

- `elif`: Önceki koşul gerçekleşmediyse başka bir koşulu kontrol eder.
- `[0]`: Listenin ilk elemanını seçer.
- `!=`: Eşit değildir anlamına gelir.

Yani kayıtlı dosyanın yolu ile kullanıcının şu anda seçtiği dosyanın yolu farklıysa program uyarı verir.Bunu neden yaptık? Çünkü bir dosyanın hash kaydını yanlışlıkla başka bir dosyayla karşılaştırmak istemiyoruz.

12-Hash değerlerini karşılaştırmak

```
elif hash_degeri == kayitli_bilgiler[1]:
    print("\nSONUC: Dosya degismedi.")

else:
    print("\nUYARI: Dosya degistirilmis olabilir!")
    print("Kayitli hash ile mevcut hash farkli.")
```

Burada iki hash değerini karşılaştırıyoruz.

- ==: Eşit mi diye kontrol eder.
- !=: Eşit değil mi diye kontrol eder.
- `elif`: Hash değerleri eşitse başarı mesajını gösterir.
- `else`: Hash değerleri farklıysa uyarı verir.

İki olasılık var:

Hash değerleri aynı

Program `SONUC: Dosya degismedi.` mesajını verir. Bu, kayıtlı değere göre bir değişiklik tespit edilmediği anlamına gelir.

Hash değerleri farklı

Program `UYARI: Dosya degistirilmis olabilir!` mesajını verir. Dosyanın içeriği değişmiş olabilir veya farklı bir durum nedeniyle hash sonucu farklılaşmış olabilir.

Küçük ama önemli bir güvenlik notu: Bu program, hash kaydının güvenilir olduğunu varsayıyor. Birisi hem dosyayı hem de `hash_kaydi.txt` dosyasını değiştirebilirse bu kontrol yanıltılabilir. Gerçek sistemlerde referans hash değerini güvenilir ve korunan bir yerde tutmak gerekir.